



Project Number 101120990

“MR-MH Architecting Tools” Technical Cookbook

**Version 0.3
08 October 2025
<Early Prototype>**

Partners and Subgrant Partners Only Distribution

University of Bremen

Project Partners: AEGIS, ATB, CEA, Centro Ricerche Fiat, Circular Economy Foundation, E-TERRY, F6S, FORTH, Harokopio University of Athens, IFADO, Jade University, KUKA Assembly & Test, Netcompany-Intrasoft, NTUA, PROFACTOR, Technology Transfer Systems, The Open Group, University of Bremen, University of York

Every effort has been made to ensure that all statements and information contained herein are accurate, however the SOPRANO Project Partners accept no liability for any error or omission in the same.

© 2025 Copyright in this document remains vested in the SOPRANO Project Partners.

DOCUMENT CONTROL

Version	Status	Date
0.1	Initial ToC draft	11 June 2025
0.2	Initial content draft	06 July 2025
0.3	Early Prototype version	08 October 2025

TABLE OF CONTENTS

1. Introduction	1
1.1 Purpose and Scope of the Cookbook	1
1.2 MAT Overview	1
1.3 Role in the SOPRANO Framework	1
2. Functional and Technical Description	2
2.1 Core Functionality	2
2.2 Architecture and Design Patterns	2
2.3 Interfaces and Interactions	4
2.3.1 Input interfaces	4
2.3.2 Output interfaces	4
2.3.3 Communication protocols	4
3. Integration Scenarios	6
3.1 Standalone Execution	6
3.2 Integration with Other SOPRANO Components	9
4. Installation and Deployment Guide	11
4.1 System Requirements	11
4.2 Installation Instructions	11
4.3 Deployment and Initialization	11
5. User Manual	16
5.1 Tool Interfaces and API	16
6. Maintenance, Support and Roadmap	16
6.1 Tool Maturity and Status	16
6.2 Future Plans and Extensions	16
7. References	18
8. Acronyms and Glossary	19
9. Appendix A: Sample Code and Scripts	20

TABLE OF FIGURES (OPTIONAL)

Figure 1: The artefact generation pipeline for the FloorPlan DSL and the scenery_builder	3
Figure 2: Updates to the pipeline for the SOPRANO components	4
Figure 3: An example of a generated 3D mesh	9

EXECUTIVE SUMMARY

This document contains instructions to set up and deploy the components for the MH-MR Architecting Tools (MAT) for the SOPRANO Open Call. For a live version of the contents of this document, please consult the official [documentation](#).

1. INTRODUCTION

This document contains instructions to set up and deploy the components for the MH-MR Architecting Tools (MAT).

1.1 PURPOSE AND SCOPE OF THE COOKBOOK

The SOPRANO Technical Cookbook is a guide for documenting each tool developed in the SOPRANO project. The goal of the cookbook is for the component owners to provide clear and complete instructions so that the awarded projects in the OC can understand, use, and connect the tool to their own systems.

The cookbook covers everything someone would need to know to work with a SOPRANO tool. This includes a basic introduction to the tool, how it works, how to install it, how to connect it to other tools, how to use it (with or without a user interface), and what support or updates are planned. If the tool offers APIs or a web interface, the author is asked to include examples, screenshots, and links to extra resources like Swagger or Postman files.

1.2 MAT OVERVIEW

The MAT components enable the modelling of indoor floorplans and the generation of execution artefacts for simulation or real-world environments. The input is the specification of the indoor environment modelled using the FloorPlan Domain-specific Language.

1.3 ROLE IN THE SOPRANO FRAMEWORK

The Composable Models at the core of the MAT tools, as its name suggests, enable the composition of different models and their semantics to be used by the tools to support users in the flexible and reconfigurable development and validation of their platforms. Generally, the MAT components are used at design-time for the modelling, composition and generation of the execution artefacts; the FloorPlan Gazebo Plugins enable the execution at run-time of dynamic behaviours for doors specified in the FloorPlan models.

2. FUNCTIONAL AND TECHNICAL DESCRIPTION

2.1 CORE FUNCTIONALITY

The MAT available in the open call include several languages and tools to model indoor buildings and generate the executable artefacts for simulation and real-world validation. The list of relevant tools is below:

- The [FloorPlan and Variation DSL](#) – Domain-specific languages to model and vary indoor environments. Modelling is a design-time activity.
- The [Scenery Builder](#) – The tool composes the JSON-LD models into a graph and enables the semantic queries of the models. It provides a command-line interface to generate artefacts for the execution of scenarios in simulation and real-world environments. The artefact generation is also a design-time tool.
- [Floor plan Gazebo plugins](#) – A set of plugins to add dynamic behaviours to doors in the Gazebo simulator. This uses the artefacts generated by the scenery builder and actuates the doors at runtime.
- [Scenery Builder Kafka interface](#) – A Kafka interface to the SOPRANO knowledge base. Other SOPRANO components can add FloorPlan models, the Kafka interface triggers the scenery builder artefact generation, and publishes the path where the generated artefacts are stored for the component to use.

2.2 ARCHITECTURE AND DESIGN PATTERNS

The FloorPlan and Variation DSL are used to model the indoor environment. These models are transformed into their Composable Model representation in JSON-LD. The resulting model is given as input to the scenery builder, potentially together with other manually-defined composable models, to generate the necessary execution artefacts. In this generation process, the Gazebo artefacts can include the specification of dynamic behaviours for certain objects (such as doors) based on the model specifications and the parametrization of the artefact generation. If that is the case; at run-time the FloorPlan Gazebo Plugins will execute the specified behaviours.

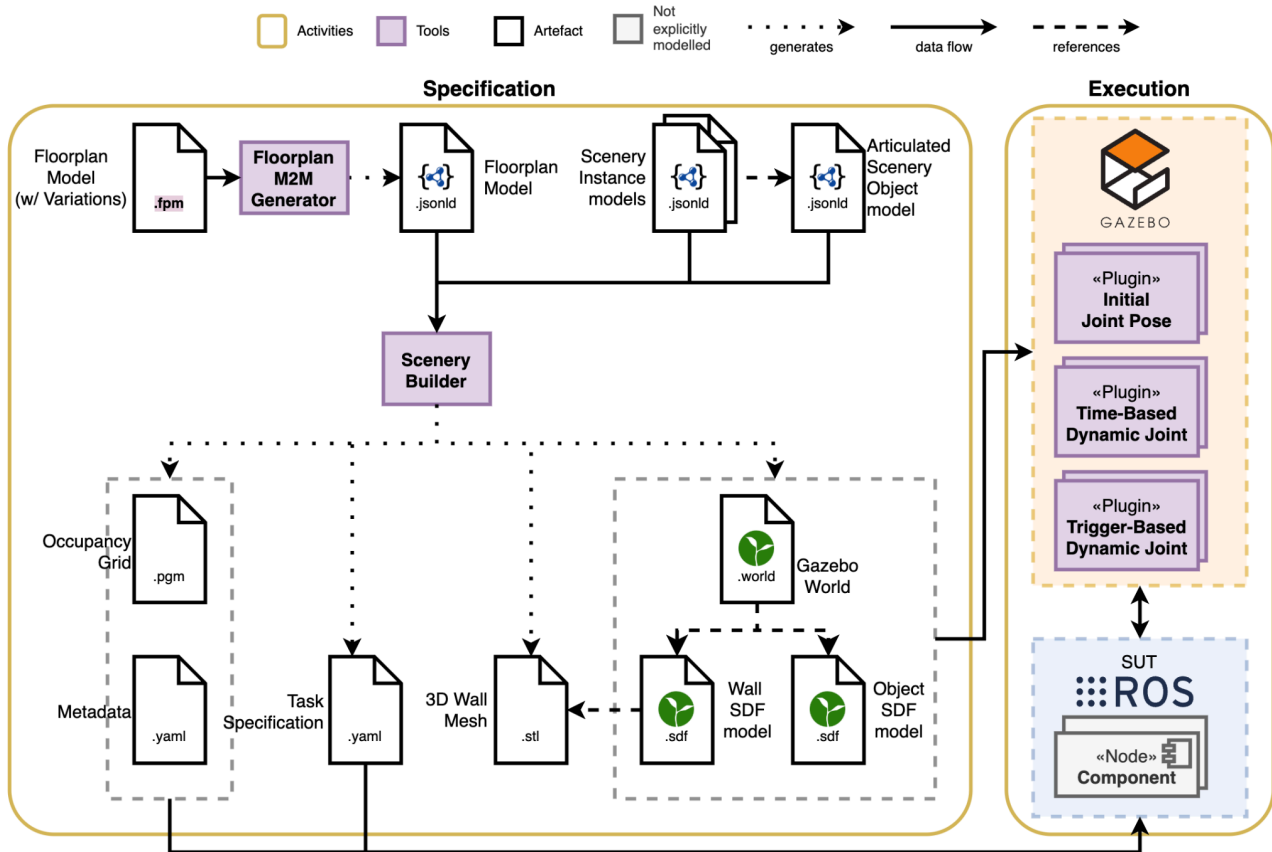


Figure 1: The artefact generation pipeline for the FloorPlan DSL and the scenery_builder

In SOPRANO, the pipeline is being expanded to support additional input (IFC files) and output (GLTF, DTS, polyline) formats. Note that at the time of release of the Open Call, not all components are available. Communication to other SOPRANO components is done using Kafka and stored in the SOPRANO Knowledge Base, as shown below:

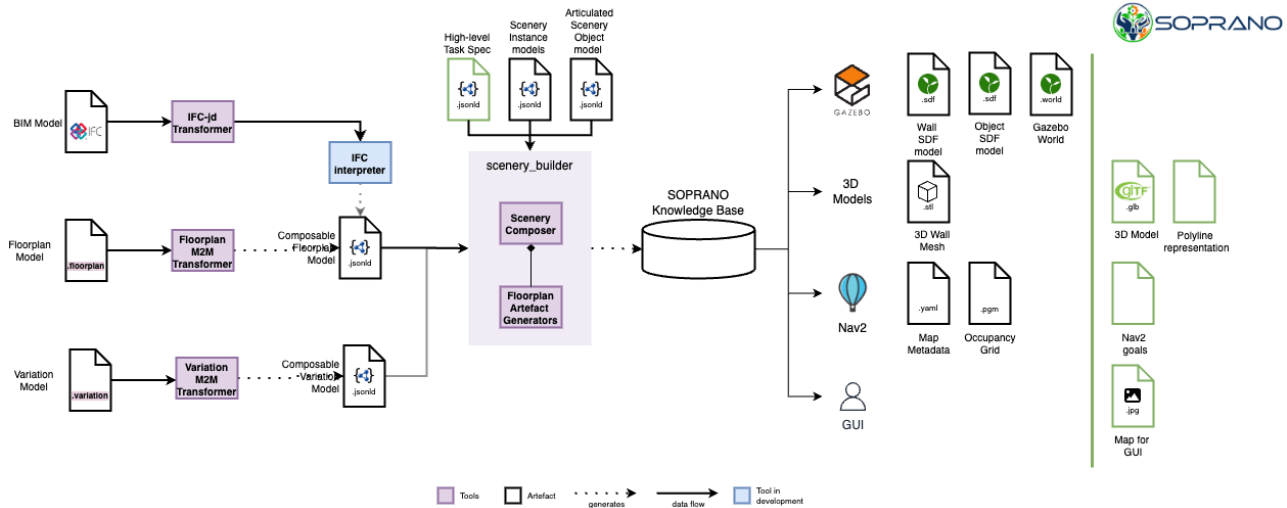


Figure 2: Updates to the pipeline for the SOPRANO components

2.3 INTERFACES AND INTERACTIONS

2.3.1 Input interfaces

A FloorPlan Model specified using the FloorPlan DSL. A tutorial for modelling an indoor environment can be found [here](#).

2.3.2 Output interfaces

Most artefacts generated by the scenery builder are not specific to particular components in SOPRANO, and can be generally used independently by any ROS system.

The generate command currently generates the following artefacts:

- 3D mesh in `.stl` format which can be used by SBT (SC08)
- Gazebo models and worlds (`.sdf`, `.config`), which can be used by SBT (SC08)
- Launch files for ROS1 and/or ROS2 (`.launch`)
- Occupancy grid map for the ROS `map_server` (`.pgm` and `.yaml`), and a `.jpg` version used by AVL (SC17)
- Tasks with a list of waypoints in `.yaml`
- A 3D polyline representation for the SOPRANO project (`.poly`) to be used by the NVL (SC03) component

2.3.3 Communication protocols

The SceneryBuilder Kafka interface, as the name suggests, utilizes Kafka to interact with the SOPRANO Knowledge Base. The interface consumes the `floorplan-model` topic (sent by the AVL component) and publishes in the `scenery-artefacts` topic with the path of the zip file containing all the generated artefacts.

3. INTEGRATION SCENARIOS

3.1 STANDALONE EXECUTION

In a standalone pipeline, the scenery builder can generate the execution artefacts through its CLI. In this process, users give the path to the FloorPlan model and the path where they wish to store the outputs.. The CLI allows users to select which subset of artefacts they wish to generate by using (or chaining) individual subcommands. The generation can be parametrized by passing the relevant (optional) arguments to each subcommand or by using a configuration file. Users can then integrate the generated artefacts into their own workflows.

Prerequisites

A FloorPlan model (fpm) of an indoor environment.

Walkthrough

The scenery builder offers a CLI to generate the execution artefacts. The usage of the generate command is as follows:

```
floorplan generate [OPTIONS] COMMAND1 [ARGS]... [COMMAND2
[ARGS]...]...
```

Where the COMMANDs correspond to one or more of:

- [door-keyframes](#) - Generate the sampled keyframes for doors with time-based behaviours
- [gazebo](#) - Generate Gazebo world, models and launch files
- [mesh](#) - Generate a 3D-mesh in STL or glTF 2.0 format
- [occ-grid](#) - Generate the occupancy grid map of the floorplan
- [polyline](#) - Generate a 3D polyline representation of the floorplan
- [tasks](#) - Generate disinfection tasks for each room in the floorplan

Each COMMAND can be parametrized by its own ARGS. In the CLI, ARGs for a COMMAND must be passed right after it has been invoked, for example:

```
floorplan generate --input-path models/json-ld --output-path
gen/model task --dist-to-corner 0.6 gazebo --ros-version ROS2 mesh
```

For more information on the parameters that can be used to customize the generation simply call:

```
floorplan generate [COMMAND] --help
```

To simplify the parametrization, the generate command can use the `--config` argument. For guidance on how to write a config file, please see [the documentation](#). An example of the supported options can be found [here](#). The equivalent command with the corresponding config file would be:

```
floorplan generate --config config.toml task gazebo mesh
```

If you have specified [dynamic doors](#) in your model, make sure you have compiled the FloorPlan Gazebo Plugins. If you are not using ROS, make sure you have added the Gazebo plugin path to GZ_SIM_SYSTEM_PLUGIN_PATH. If you are using ROS, make sure you have sourced your colcon workspace. Finally, make sure to add the paths for the generated 3d-mesh and gazebo to your GZ_SIM_RESOURCE_PATH. If you have any doors with timed-based behaviours, make sure to add the path the doors/behaviours path to GZ_SIM_RESOURCE_PATH as well.

Basic Demo

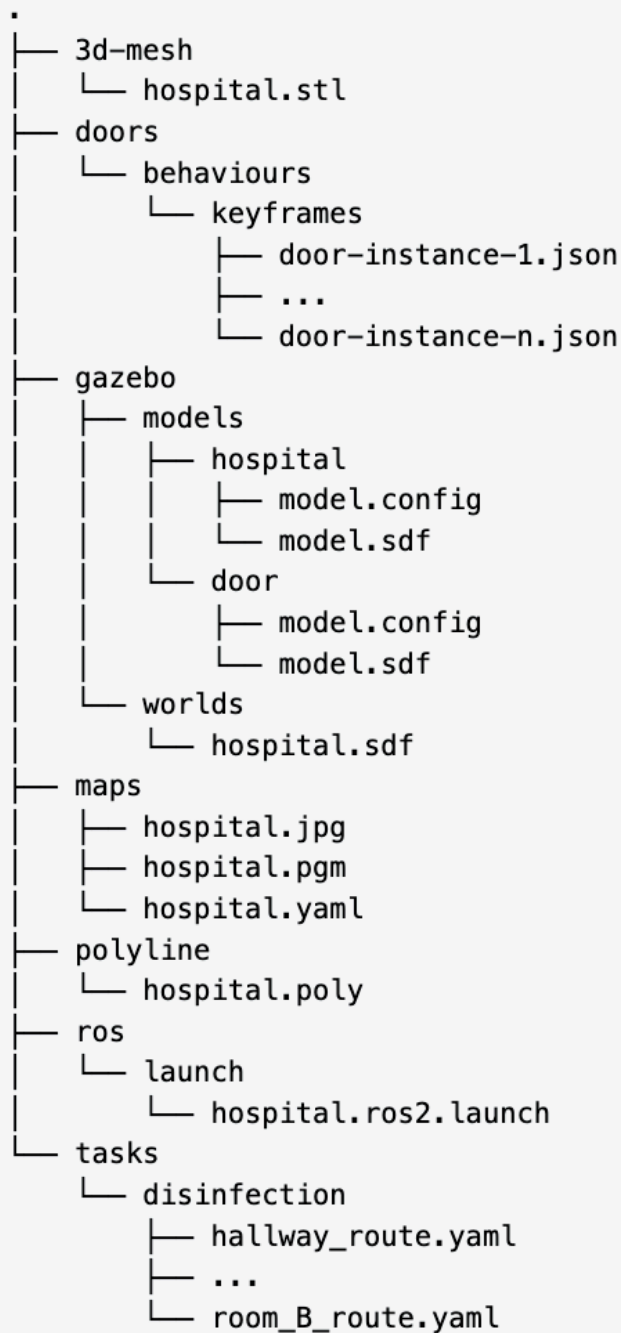
Let us use an existing FloorPlan model: [hospital.fpm](#). After downloading it, we generate its composable representation:

```
floorplan transform --model hospital.fpm --output-path
gen/hospital/json-ld
```

We will now generate all the artefacts, and parametrize the waypoints in the tasks to be 1.2m from the wall and the laser height of our robot at 0.3m to generate the occupancy grid:

```
floorplan generate -i gen/hospital/json-ld -o gen/hospital mesh
gazebo tasks --dist-to-corner 1.2 occ-grid --laser-height 0.3
door-keyframes polyline
```

Below you can find all the artefacts generated by the scenery_builder and their default output path:



Next, we export the necessary paths to GZ_SIM_RESOURCE_PATH

```

export
GZ_SIM_RESOURCE_PATH=$HOME/mymodels/gen/hospital/gazebo/worlds:$HOME/mymodels/gen/hospital/gazebo/models:$HOME/mymodels/gen/hospital/3d-mesh:$GZ_SIM_RESOURCE_PATH
    
```

Finally, open the Gazebo world directly. First, start the server:

```
gz sim -s gen/hospital/gazebo/worlds/hospital.sdf
```

Then the Gazebo GUI:

```
gz sim -g
```

The result should look something like the figure below:

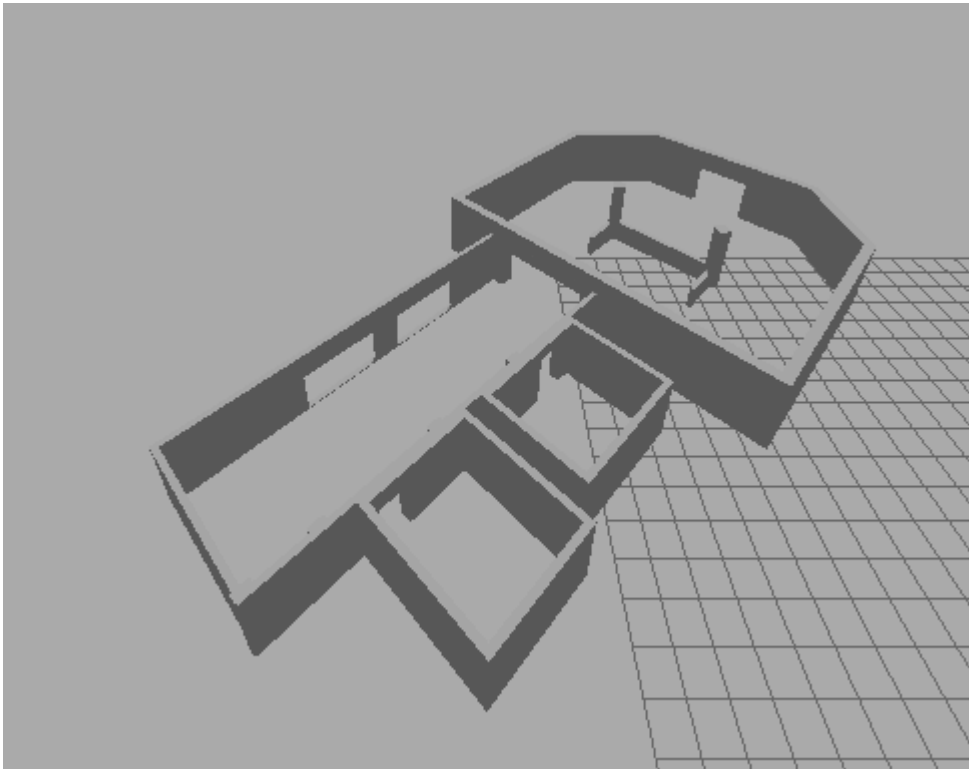


Figure 3: An example of a generated 3D mesh

3.2 INTEGRATION WITH OTHER SOPRANO COMPONENTS

To interact with other SOPRANO components through the Knowledge Base, the Scenery Builder Kafka interface must be configured. The scenery builder consumes the floorplan-model sent by the AVL component, and publishes the path to the generated artefacts in the scenery-artefacts topic.

The deployment of this component requires the Knowledge Base, Kafka and AVL components to be deployed, but not necessarily in the same machine as the MAT and Scenery Builder Kafka Interface. The Scenery Builder Kafka interface runs continuously. Any time a new model is published to floorplan-model, it triggers the artefact generation.

Configuration

Create an .env file to store your credentials and the relevant API endpoints to the Knowledge Base and the Kafka server, for example:

```
KAFKA_USER=myuser
```

```
KAFKA_PASS=12345pass
```

KAFKA_URL=kafka-url:9000

REST_UPLOAD_ARTEFACT_URL=http://server-url:8000/api/upload/artefact

4. INSTALLATION AND DEPLOYMENT GUIDE

4.1 SYSTEM REQUIREMENTS

Minimum:

- Ubuntu 24.04
- Docker

For the FloorPlan Gazebo Plugins

- Gazebo Harmonic (and optionally ROS2 Jazzy)

Optional:

For the local installation of the components:

- Python 3.11, pip
- Git
- Blender v4.4.1 or bpy 4.4.1

4.2 INSTALLATION INSTRUCTIONS

Installation instructions for the standalone usage can be found in the [README of the scenery_builder](#) and the [component documentation](#). Docker is the recommended way to use this component for basic usage. The local installation is only recommended for advanced users and integration with in-house tools.

Optionally, for the integration with other SOPRANO components, the [MH-MR Architecting tools](#) contains the installation instructions for the Knowledge Base and Kafka interface, and installs all other necessary components (including the FloorPlan DSL and the scenery_builder)..

4.3 DEPLOYMENT AND INITIALIZATION

Scenery Builder

CLI

The documentation of the commands and arguments can be seen in the terminal by passing the `--help` argument to any command. The detailed documentation of CLI can also be found online [here](#). For the sake of simplicity, we will only cover the configuration file here.

How to create a config file

The config file usage is documented [here](#). The config file must be written using [TOML](#), and follow the nesting structure of the commands and their arguments. For example, to generate the tasks for a given model, and parametrize the distance to 0.9 m, one would run:

```
floorplan generate --inputs model/json-ld --outputs  
gen/model/tasks tasks --dist-to-corner 0.9
```

The same result can be achieved using the following config file:

```
[generate]

inputs = "model/json-ld"

outputs = "gen/model/tasks"


[generate.tasks]

dist_to_corner = 0.7
```

Note how the options are scoped to the command being used (i.e, `generate` has the options `inputs` and `outputs`). To add arguments for subcommands, simply reference the subsection in the form `command.subcommand`. Now to run the command, we pass the `--config` option to the generate command instead:

```
floorplan generate --config config.toml tasks
```

A full configuration file is shown below:

```
[generate]

inputs = /path/to/json-ld/

outputs = /output/path


[generate.tasks]

dist_to_corner = 0.7


[generate.gazebo]

ros_version = "ROS2"

ros_pkg = "floorplan_models"
```



```
[generate.occ_grid]
border = 50
# generation metadata
laser_height = 0.7
# map_server required metadata
resolution = 0.05
occupied_threshold = 0.65
free_threshold = 0.196
negate = 0
unknown_value = 200
occupied_value = 0
free_value = 255
```

```
[generate.door_keyframes]
start_frame = 0
end_frame = 180
start_state = 0.0
sampling_interval = 30
state_change_probability = 0.5
```

Docker

For a more detailed version, please see the [Usage](#) instructions in the documentation.

To use the scenery_builder via Docker, simply mount the input and output paths as volumes and run the container. This will run the floorplan generate command with the default values defined in the entrypoint.

```
docker run -v <local input path>:/usr/src/app/models -v <local
output path>:/usr/src/app/output scenery_builder:latest
```

If you want to change the path of the volumes inside the Docker container (e.g., change `/usr/src/app/models` or `/usr/src/app/output` to something else) or want to customize the arguments, use the following:

```
docker run -v <local input path>:/usr/src/app/models -v <local
output path>:/usr/src/app/output scenery_builder:latest -i
/usr/src/app/models -o /usr/src/app/output <optional arguments>
```

If you are using a config file, make sure to add it or copy it into the container, and to use the path in the container in the `--config` argument.

FloorPlan Gazebo Plugins

Export the necessary paths to `GZ_SIM_RESOURCE_PATH`. For a static environment:

```
export GZ_SIM_RESOURCE_PATH=$HOME/path/to/models/gen/<model
name>/gazebo/worlds:$HOME/path/to/models/gen/<model
name>/gazebo/models:$HOME/path/to/models/gen/<model
name>/3d-mesh:$GZ_SIM_RESOURCE_PATH
```

For dynamic doors with timed-based behaviours:

```
export GZ_SIM_RESOURCE_PATH=$HOME/path/to/models/gen/<model name>
/gazebo/worlds:$HOME/path/to/models/gen/<model
name>/gazebo/models:$HOME/path/to/models/gen/<model
name>/3d-mesh:$HOME/path/to/models/gen/<model
name>/doors/behaviours:$GZ_SIM_RESOURCE_PATH
```

Gazebo

Export the path to the build folder of the plugins:

```
export
GZ_SIM_SYSTEM_PLUGIN_PATH=/path/to/floorplan_gazebo_plugins/build
```

Start the server:

```
gz sim -s <path to generated artefacts>/gazebo/worlds/<model
name>.sdf
```

Then the Gazebo GUI:

```
gz sim -g
```

ROS

Assuming that you’ve used an existing ROS <package> as the --output-path of the scenery builder, and you have used the --ros-pkg argument in the generate gazebo command to customize the launch files to said <package>, you can use the generated launch files:

```
ros2 run <package> <model name>.ros2.launch
```

Scenery Builder Kafka Interface

It is recommended to use the Docker image for ease of deployment.

In the Docker image, the default CMD is already set for the Kafka consumer, so creating a container will run the interface automatically. You should use the argument --env-file to pass the path of the .env file containing the credentials to login and URLs to docker run, for example:

```
docker run --env-file .env  
ghcr.io/secorolab/soprano\_scenery\_kafka:latest
```

The Dockerfile uses the images of the development branch of the two floorplan components. The images are built as part of the CI and hosted on GitHub.

5. USER MANUAL

5.1 TOOL INTERFACES AND API

Scenery Builder

The command line interface is documented [here](#).

Scenery Builder Kafka Interface

Kafka streams:

- /floorplan-model (consumer)
 - Key: scenery_id – unique ID of the scenery model
 - Value: url – URL where the FPM model is stored
- /scenery-artefacts (producer)
 - key: scenery_id – unique ID of the scenery model
 - value: url – Where the zip file has been stored
 - Headers
 - title: <scenery_id> environment – Descriptive title for the GUI
 - description: description – Human-readable description of what the model is
 - use_case: use_case – The use case or project the model is for
 - content-type: application/zip

6. MAINTENANCE, SUPPORT AND ROADMAP

6.1 TOOL MATURITY AND STATUS

Component	Maturity	Technology Readiness Level	Known limitations
FloorPlan DSL	Beta	TRL 6	
Scenery Builder	Alpha	TRL 5	
FloorPlan Gazebo Plugins	Prototype	TRL 4	
Scenery Builder Kafka Interface	Prototype	TRL 3	

6.2 FUTURE PLANS AND EXTENSIONS

Upcoming features:

- Generation of the 3D-mesh in GLTF format for the HDT (SC09) simulator
- Using Building Information Modelling (BIM) files (specifically IFC files) as inputs in the scenery builder

Planned refactoring or interface changes:

- Language extensions for doors and other building elements in the FloorPlan DSL
- Additional arguments in the Scenery Builder CLI

Planned extensions to support other SOPRANO tools:

- Generation of artefacts for the HDT (SC09) simulator
- Generation of navigation goals based on/with respect to room features (e.g. doors, power outlets, etc)

7. REFERENCES

8. ACRONYMS AND GLOSSARY

API	Application Programming Interface
DSL	Domain Specific Language
CLI	Command Line Interface
BIM	Building Information Modelling
HDT	Human Digital Twin
CI	Continuous Integration
IFC	Industry Foundation Classes
GUI	Graphical User Interface
AVL	Advanced Visualization

9. **APPENDIX A: SAMPLE CODE AND SCRIPTS**

For tutorials, examples and scripts please see the official [documentation](#).